

SQL

Des bases a la preparation d'entretien technique

Requetes, jointures, agregations, fonctions fenetrees,
CTE et index – avec la syntaxe PostgreSQL.

Cours a imprimer – support d'entrainement

Table des matières

1	Introduction	2
2	Types de donnees	2
2.1	Convertir un type avec CAST	3
3	Creer et peupler des tables	4
4	Selectionner des donnees	4
4.1	Astuces de manipulation	5
5	Jointures	7
5.1	Plusieurs jointures a la fois	8
6	Agregations et regroupement	9
6.1	Arrondir les nombres	9
7	Sous-requetes	10
8	CTE – Common Table Expressions	11
9	Fonctions fenetrees (window functions)	11
10	Types de jointure des ensembles	12
11	Index et performance	12
12	Transactions	13
13	Erreurs et pieges frequents	13
14	Fiche recapitulative	14

1. Introduction

SQL (*Structured Query Language*) est le langage utilisé pour interroger et manipuler des données stockées dans une base de données *relationnelle* : des tables composées de lignes et de colonnes, reliées entre elles par des *cles*. Contrairement à un langage comme Python, SQL est *declaratif* : on décrit *quel* résultat on veut, pas *comment* l'obtenir – c'est le *moteur* de la base (le *query planner*) qui choisit la meilleure stratégie d'exécution.

Les exemples de ce cours utilisent la syntaxe **PostgreSQL**. L'essentiel est du SQL standard (norme ANSI) et fonctionne tel quel sur la plupart des moteurs (MySQL, SQL Server, SQLite) ; les différences notables sont signalées dans des encadres.

Les grandes familles de commandes

DDL (*Data Definition Language*) : CREATE, ALTER, DROP – définissent la structure. **DML** (*Data Manipulation Language*) : SELECT, INSERT, UPDATE, DELETE – manipulent les données. **DCL** (*Data Control Language*) : GRANT, REVOKE – gèrent les droits d'accès.

2. Types de données

Chaque colonne d'une table a un *type* fixe, choisi à la création. Le type détermine ce que le SGBD accepte d'y stocker, l'espace occupé sur disque, et les opérations autorisées (on ne peut pas faire de moyenne sur du texte). Bien choisir ses types dès le départ évite des migrations coûteuses plus tard.

Categorie	Types courants (PostgreSQL)
Entiers	SMALLINT (16 bits), INTEGER (32 bits), BIGINT (64 bits). SERIAL/BIGSERIAL : entier auto-incrémenté, pratique pour une clé primaire.
Nombres décimaux	NUMERIC(precision, echelle) : précision exacte, pour de l'argent. REAL, DOUBLE PRECISION : virgule flottante approximative, pour du calcul scientifique.
Texte	TEXT : longueur illimitée. VARCHAR(n) : longueur limitée à n. CHAR(n) : longueur fixe, complète avec des espaces (rarement utile).
Booleen	BOOLEAN : TRUE, FALSE, ou NULL (trois états, pas deux).
Date et heure	DATE (jour seul), TIME (heure seule), TIMESTAMP (date + heure, sans fuseau), TIMESTAMPTZ (date + heure, avec fuseau – recommande des que l'app a des utilisateurs dans plusieurs fuseaux).
Identifiants	UUID : identifiant unique global, utile pour éviter des ID prédictibles ou fusionner des données de plusieurs sources.
Structures	JSON/JSONB : données semi-structurées (JSONB est stocké en binaire, indexable, généralement préféré). ARRAY : tableau d'un type donné (INTEGER[], TEXT[]).

```
CREATE TABLE produits (
  id          BIGSERIAL PRIMARY KEY,
  nom         VARCHAR(120) NOT NULL,
  description TEXT,
  prix        NUMERIC(10, 2) NOT NULL,      -- 10 chiffres, dont 2 après la virgule
  en_stock    BOOLEAN          DEFAULT TRUE,
  tags        TEXT[],           -- tableau de textes
  attributs   JSONB,            -- données libres, indexables
  cree_le     TIMESTAMPTZ      DEFAULT NOW()
```

```
);

-- Conversion explicite entre types avec CAST ou ::
SELECT CAST('42' AS INTEGER);
SELECT '42'::INTEGER;           -- syntaxe raccourcie propre à PostgreSQL
```

2.1 Convertir un type avec CAST

`CAST(valeur AS type)` force la conversion explicite d'une valeur vers un autre type – l'équivalent SQL de `int(...)/str(...)` en Python. Utile des qu'une colonne texte contient en réalité des nombres ou des dates (import brut, champ `TEXT` mal type à l'origine), ou pour garantir un type précis dans un calcul (voir le piège sur la division entière, section 5).

```
-- Texte vers nombre, et nombre vers texte
SELECT CAST('123' AS INTEGER);           -- 123
SELECT CAST(19.99 AS TEXT);              -- '19.99'
SELECT '19.99'::NUMERIC;                 -- 19.99 (syntaxe :: PostgreSQL)

-- Texte vers date, très courant sur des données importées
SELECT CAST('2024-03-15' AS DATE);
SELECT '2024-03-15'::DATE + 7;           -- arithmétique de dates après conversion

-- Forcer un type avant une division pour éviter la division entière
SELECT CAST(total AS NUMERIC) / nb_articles AS moyenne
FROM commandes;

-- CAST dans une condition : comparer une colonne TEXT mal typée à un nombre
SELECT * FROM logs WHERE CAST(code_erreur AS INTEGER) > 400;

-- CAST implicite silencieux : PostgreSQL convertit parfois sans CAST explicite
SELECT '10' = 10;           -- fonctionne (conversion implicite), mais à éviter :
                             -- moins lisible et empêche souvent l'usage d'un index
```

Piège courant

`CAST` **échoue** (lève une erreur qui interrompt la requête) si la valeur n'est pas convertible : `CAST('abc' AS INTEGER)` plante toute la requête, même si une seule ligne sur un million pose problème. Sur PostgreSQL, utiliser `value::type` n'échappe pas à ce risque (même comportement que `CAST`). Pour convertir sans faire planter la requête entière sur des données sales, filtrer au préalable les valeurs invalides (par exemple avec une expression régulière via `~` ou `SIMILAR TO`), ou utiliser une fonction de conversion tolérante spécifique au moteur si elle existe.

NUMERIC vs FLOAT

`NUMERIC` stocke une valeur *exacte* (aucune erreur d'arrondi) mais est plus lent ; c'est le choix par défaut pour de l'argent. `REAL/DOUBLE PRECISION` sont des flottants binaires, plus rapides mais avec des approximations (`0.1 + 0.2` peut ne pas donner exactement `0.3`) – à réserver aux calculs où une petite imprecision est acceptable.

Piège courant

`VARCHAR(n)` sans `n` et `TEXT` sont strictement équivalents en PostgreSQL (aucune différence de performance) : la limite de longueur n'apporte qu'une validation, pas un gain de vitesse. Ne pas confondre avec MySQL, où `VARCHAR` et `TEXT` sont stockés différemment.

3. Créer et peupler des tables

Une table est définie par ses colonnes, chacune avec un *type* et d'éventuelles *contraintes*. Les contraintes empêchent les données invalides d'entrer dans la base – c'est la première ligne de défense de l'intégrité des données, avant même le code applicatif.

```
CREATE TABLE clients (  
  id          SERIAL PRIMARY KEY,          -- entier auto-incremente  
  nom         TEXT NOT NULL,  
  email       TEXT UNIQUE NOT NULL,  
  age         INTEGER CHECK (age >= 0),  
  cree_le     TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE commandes (  
  id          SERIAL PRIMARY KEY,  
  client_id   INTEGER NOT NULL REFERENCES clients(id), -- cle etrangere  
  montant     NUMERIC(10, 2) NOT NULL,  
  statut      TEXT DEFAULT 'en_attente'  
);  
  
INSERT INTO clients (nom, email, age)  
VALUES ('Alice', 'alice@mail.com', 30),  
      ('Bob', 'bob@mail.com', 25);  
  
UPDATE clients SET age = 31 WHERE nom = 'Alice';  
  
DELETE FROM clients WHERE email = 'bob@mail.com';
```

Piege courant

DELETE FROM table sans clause WHERE supprime **toutes** les lignes. UPDATE table SET ... sans WHERE modifie **toutes** les lignes. Toujours verifier la clause WHERE avant d'exécuter, surtout en production – idéalement, tester d'abord avec un SELECT utilisant la meme condition.

4. Selectionner des donnees

SELECT est la commande la plus utilisée : elle extrait des lignes d'une ou plusieurs tables. L'ordre *logique* d'exécution d'une requête n'est pas l'ordre dans lequel on l'écrit – comprendre cet ordre évite beaucoup d'erreurs (par exemple, pourquoi un alias défini dans SELECT n'est pas utilisable dans WHERE).

Ordre logique d'exécution

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT.
WHERE filtre les lignes *avant* regroupement ; HAVING filtre les groupes *apres*.

```
SELECT nom, age  
FROM clients  
WHERE age >= 18 AND nom ILIKE 'a%'          -- ILIKE = LIKE insensible a la casse  
ORDER BY age DESC  
LIMIT 10;  
  
-- Colonnes calculees et alias  
SELECT nom, age, age >= 18 AS majeur  
FROM clients;
```

```
-- Valeurs distinctes
SELECT DISTINCT statut FROM commandes;

-- Tester une valeur NULL : jamais avec = ou !=
SELECT * FROM clients WHERE email IS NOT NULL;

-- Plage de valeurs et liste de valeurs
SELECT * FROM clients WHERE age BETWEEN 18 AND 65;
SELECT * FROM commandes WHERE statut IN ('en_attente', 'expediee');
```

Piege courant

NULL represente une valeur *inconnue*, pas zero ni chaine vide. `colonne = NULL` ne renvoie jamais TRUE (meme si la colonne est vraiment NULL) : il faut `IS NULL` ou `IS NOT NULL`.

4.1 Astuces de manipulation

Au-dela du simple filtrage, quelques fonctions et operateurs reviennent sans cesse pour *extraire*, *nettoyer* ou *transformer* une colonne directement dans la requete – sans avoir a rapatrier les donnees brutes pour les retravailler cote application.

```
-- Debut / fin d'une chaine : LEFT et RIGHT prennent un nombre de caracteres
SELECT LEFT(nom, 3) AS debut,      -- 3 premiers caracteres
       RIGHT(nom, 3) AS fin        -- 3 derniers caracteres
FROM clients;

-- SUBSTRING : extraire depuis une position, sur une longueur donnee
SELECT SUBSTRING(email FROM 1 FOR 5) AS debut_email,
       SUBSTRING(email FROM POSITION('@' IN email) + 1) AS domaine
FROM clients;

-- LENGTH : longueur d'une chaine (utile combine a LEFT/RIGHT pour du dynamique)
SELECT nom, LENGTH(nom) AS longueur FROM clients;

-- Nettoyage : equivalents SQL de strip/lstrip/rstrip en Python
SELECT TRIM(nom), LTRIM(nom), RTRIM(nom) FROM clients;
SELECT TRIM(BOTH '#' FROM '##titre##'); -- 'titre'

-- Casse et concatenation
SELECT UPPER(nom), LOWER(email) FROM clients;
SELECT nom || ' <' || email || '>' AS contact FROM clients; -- || = concatenation
SELECT CONCAT(nom, ' <', email, '>') AS contact FROM clients; -- equivalent, gere les
NULL

-- Remplacement
SELECT REPLACE(email, '@mail.com', '@nouveau.com') FROM clients;

-- COALESCE : premiere valeur non-NULL dans une liste -- LA methode pour
-- gerer les valeurs manquantes sans passer par un CASE WHEN
SELECT nom, COALESCE(tel, 'non renseigné') AS tel FROM clients;

-- NULLIF : renvoie NULL si les deux valeurs sont egales (utile pour eviter
-- une division par zero sans passer par un CASE WHEN)
SELECT total / NULLIF(quantite, 0) AS prix_unitaire FROM commandes;

-- CASE WHEN : la seule structure conditionnelle de SQL, comme un if/elif/else
SELECT nom, age,
       CASE
```

```

        WHEN age < 18 THEN 'mineur'
        WHEN age < 65 THEN 'adulte'
        ELSE 'senior'
    END AS categorie
FROM clients;

```

Filtrer sur le premier ou le dernier caractere

LEFT, RIGHT et LIKE ne servent pas qu'à afficher une colonne transformée : places dans un WHERE ou un CASE WHEN, ils permettent de *filtrer* ou de *brancher* une requête selon le début ou la fin d'une chaîne.

```

-- Filtrer sur le PREMIER caractere (ou les n premiers)
SELECT * FROM clients WHERE LEFT(nom, 1) = 'A';           -- commence par 'A'
SELECT * FROM clients WHERE nom LIKE 'A%';               -- equivalent, souvent
                                                         -- plus lisible et
                                                         -- mieux optimise

-- Filtrer sur le DERNIER caractere (ou les n derniers)
SELECT * FROM fichiers WHERE RIGHT(nom, 4) = '.pdf';      -- se termine par '.pdf'
SELECT * FROM fichiers WHERE nom LIKE '%.pdf';            -- equivalent avec LIKE

-- Un caractere precis n'importe ou dans la chaine : POSITION ou LIKE avec %
SELECT * FROM clients WHERE POSITION('@' IN email) > 0;   -- contient un '@'
SELECT * FROM clients WHERE email LIKE '%@%';            -- equivalent

-- Combiner debut ET fin dans une seule condition
SELECT * FROM references
WHERE LEFT(code, 2) = 'FR' AND RIGHT(code, 1) = 'X';

-- Brancher un calcul selon le premier caractere (ex. categoriser un code postal)
SELECT ville, code_postal,
    CASE
        WHEN LEFT(code_postal, 2) = '75' THEN 'Paris'
        WHEN LEFT(code_postal, 2) IN ('92', '93', '94') THEN 'Petite couronne'
        ELSE 'Autre'
    END AS zone
FROM adresses;

```

LIKE vs LEFT/RIGHT pour filtrer

Pour un test de prefixe ou suffixe simple, colonne LIKE 'A%' est en general prefere a LEFT(colonne, 1) = 'A' : sur une colonne **indexee**, le moteur peut utiliser l'index avec un motif LIKE qui commence par un prefixe fixe ('A%'), alors qu'appliquer une fonction comme LEFT sur la colonne empeche generalement l'index d'etre utilise (voir le piege sur les index, section 10). LIKE '%A' (suffixe) ne peut de toute facon pas utiliser un index B-tree standard, qu'on passe par LIKE ou par RIGHT.

Alignement des positions

En SQL, les positions de chaîne commencent à 1 (pas 0 comme en Python). LEFT(s, n) et RIGHT(s, n) avec un n negatif retirent n caracteres depuis l'autre bout (LEFT('bonjour', -2) donne 'bonj') – pratique mais facile a oublier.

Piege courant

COALESCE et CASE WHEN s'arrêtent à la première condition vraie / première valeur non-NULL : l'ordre des arguments compte. Utiliser || pour concatener avec une valeur NULL propage le NULL à toute la chaîne résultante ('a' || NULL donne NULL, pas 'a'); CONCAT n'a pas ce problème, il traite NULL comme une chaîne vide.

5. Jointures

Une jointure combine les lignes de deux tables selon une condition de correspondance (généralement une clé étrangère). C'est le mécanisme qui justifie de *normaliser* les données (les répartir dans plusieurs tables sans duplication) plutôt que de tout stocker dans une seule table plate.

```
-- INNER JOIN : seulement les lignes qui correspondent des deux cotes
SELECT c.nom, o.montant
FROM clients c
INNER JOIN commandes o ON o.client_id = c.id;

-- LEFT JOIN : toutes les lignes de gauche, même sans correspondance à droite
-- (colonnes de droite à NULL si pas de commande)
SELECT c.nom, o.montant
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id;

-- Clients SANS aucune commande (anti-jointure classique)
SELECT c.nom
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id
WHERE o.id IS NULL;

-- FULL OUTER JOIN : union des deux cotes, correspondance ou non
SELECT c.nom, o.montant
FROM clients c
FULL OUTER JOIN commandes o ON o.client_id = c.id;

-- Jointure d'une table sur elle-même (self-join) : ex. trouver les paires
-- d'employés qui partagent le même manager
SELECT e1.nom AS employe, e2.nom AS collegue
FROM employes e1
JOIN employes e2 ON e1.manager_id = e2.manager_id AND e1.id < e2.id;
```

Quelle jointure choisir ?

INNER JOIN : uniquement les correspondances (le plus courant). LEFT JOIN : tout de la table de gauche, complète par NULL si pas de correspondance – utile pour « trouver ce qui manque ». FULL OUTER JOIN : rare, sert à comparer deux ensembles en gardant tout des deux cotes.

Piege courant

Filtrer sur la table de droite d'un LEFT JOIN dans le WHERE (au lieu du ON) transforme silencieusement le LEFT JOIN en INNER JOIN : WHERE o.statut = 'payee' élimine les clients sans commande alors qu'on voulait les garder. La condition sur la table jointe doit rester dans le ON si on veut conserver les lignes sans correspondance.

5.1 Plusieurs jointures a la fois

Une requete peut enchaîner autant de JOIN que nécessaire : chaque nouvelle jointure s'applique au resultat *deja construit* par les precedentes, pas seulement a la premiere table. On peut melanger les types de jointure (INNER, LEFT...) selon ce que chaque table doit apporter.

```
-- 3 tables : clients -> commandes -> lignes_commande -> produits
SELECT c.nom, p.nom AS produit, l.quantite
FROM clients c
JOIN commandes o      ON o.client_id = c.id
JOIN lignes_commande l ON l.commande_id = o.id
JOIN produits p        ON p.id = l.produit_id;

-- Melanger INNER et LEFT : garder TOUS les clients, mais seulement les
-- commandes payees quand elles existent
SELECT c.nom, o.montant
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id AND o.statut = 'payee';
-- (la condition sur o est dans le ON, pas le WHERE : voir le piege ci-dessus)

-- Agreger APRES plusieurs jointures : total depense par client, produits inclus
SELECT c.nom, SUM(l.quantite * p.prix) AS total
FROM clients c
JOIN commandes o      ON o.client_id = c.id
JOIN lignes_commande l ON l.commande_id = o.id
JOIN produits p        ON p.id = l.produit_id
GROUP BY c.nom;
```

Piege courant

Enchaîner des jointures *one-to-many* (une commande a plusieurs lignes, une ligne peut correspondre a plusieurs produits...) **multiplie** les lignes du resultat : un client avec 2 commandes de 3 lignes chacune apparait 6 fois. Appliquer directement SUM(o.montant) sur ce resultat compte donc o.montant plusieurs fois et gonfle le total – c'est l'erreur la plus frequente des requetes a plusieurs jointures. Deux solutions : agreger *avant* de joindre (dans une sous-requete ou une CTE qui resume la table « multiple » a une ligne par cle), ou utiliser COUNT(DISTINCT ...) / SUM(DISTINCT ...) avec prudence (fonctionne seulement si les valeurs dupliquees sont identiques).

```
-- Mauvais : le montant de chaque commande est compte une fois par ligne produit
SELECT c.nom, SUM(o.montant) AS total    -- FAUX si une commande a plusieurs lignes
FROM clients c
JOIN commandes o      ON o.client_id = c.id
JOIN lignes_commande l ON l.commande_id = o.id
GROUP BY c.nom;

-- Correct : agreger le montant par commande AVANT de joindre les lignes de detail
WITH commandes_resumees AS (
    SELECT client_id, SUM(montant) AS total
    FROM commandes
    GROUP BY client_id
)
SELECT c.nom, r.total
FROM clients c
JOIN commandes_resumees r ON r.client_id = c.id;
```

Methode pour une requete a plusieurs jointures

Ajouter les tables une a la fois : apres chaque JOIN, verifier avec **SELECT *** et **COUNT(*)** que le nombre de lignes obtenu est celui attendu, *avant* d'empiler l'agregation ou la jointure suivante. C'est la seule facon fiable de reperer une multiplication de lignes avant qu'elle ne fausse un SUM/COUNT final.

6. Agregations et regroupement

Les fonctions d'agregation (COUNT, SUM, AVG, MIN, MAX) resument plusieurs lignes en une seule valeur. **GROUP BY** les applique *par groupe* plutot que sur toute la table : chaque valeur distincte des colonnes groupees devient une ligne de resultat. Toute colonne du **SELECT** qui n'est pas agregee doit apparaitre dans le **GROUP BY**.

```
-- Nombre et montant total de commandes par client
SELECT client_id, COUNT(*) AS nb_commandes, SUM(montant) AS total
FROM commandes
GROUP BY client_id;

-- HAVING filtre les groupes (apres agregation) ; WHERE filtre les lignes (avant)
SELECT client_id, SUM(montant) AS total
FROM commandes
WHERE statut = 'payee'           -- filtre les lignes avant regroupement
GROUP BY client_id
HAVING SUM(montant) > 100;      -- filtre les groupes apres regroupement

-- COUNT(*) compte les lignes ; COUNT(colonne) ignore les NULL de cette colonne
SELECT COUNT(*) AS total_lignes, COUNT(email) AS emails_renseignes
FROM clients;
```

Piege courant

WHERE ne peut pas utiliser un alias defini dans **SELECT**, ni filtrer sur le resultat d'une agregation (**WHERE COUNT(*) > 1** est invalide) : l'agregation n'existe pas encore a ce stade de l'execution logique. C'est justement le role de **HAVING**.

6.1 Arrondir les nombres

Une agregation comme **AVG** produit souvent un nombre avec beaucoup de decimales : il faut l'arrondir explicitement pour un affichage propre (un prix, un pourcentage, une moyenne). SQL offre plusieurs fonctions selon le comportement voulu – arrondir au plus proche, toujours arrondir vers le haut, ou simplement tronquer sans arrondir.

```
-- ROUND(valeur, decimales) : arrondi classique au plus proche
SELECT ROUND(19.855, 2);      -- 19.86
SELECT ROUND(19.854, 2);      -- 19.85
SELECT ROUND(1234.5, -2);     -- 1200  (decimales negatives : arrondit avant la virgule)

-- CEIL / CEILING : arrondit toujours vers le haut (utile pour de la pagination)
SELECT CEIL(4.1);              -- 5
SELECT CEIL(nb_articles / 10.0) AS nb_pages FROM catalogue;  -- pages necessaires

-- FLOOR : arrondit toujours vers le bas
SELECT FLOOR(4.9);             -- 4

-- TRUNC : coupe les decimales sans arrondir (contrairement a ROUND)
```

```
SELECT TRUNC(19.999, 2);      -- 19.99   (pas 20.00 : ne regarde pas la decimale
                               suivante)

-- Combine a une agregation : moyenne lisible a 2 decimales
SELECT client_id, ROUND(AVG(montant), 2) AS moyenne_commande
FROM commandes
GROUP BY client_id;
```

Piege courant

Diviser deux entiers en SQL effectue une **division entiere** qui tronque le resultat, comme en Python avec `//` : `SELECT 7 / 2` donne 3, pas 3.5 (sur PostgreSQL comme sur la plupart des moteurs, si les deux operandes sont de type `INTEGER`). Il faut forcer un type decimal avant la division : `7 / 2.0`, `7::NUMERIC / 2`, ou `CAST(7 AS NUMERIC) / 2` donnent bien 3.5. C'est une source frequente de resultats faux dans des calculs de pourcentage ou de moyenne.

Rappel : arrondi et stockage vont ensemble

Arrondir à l'affichage ne suffit pas si la colonne est déjà stockée en `FLOAT/REAL` (virgule flottante binaire, sujette à des erreurs d'arrondi). Pour de l'argent, la colonne elle-même doit être en `NUMERIC(10, 2)` dès la création de la table (voir la section Types de variables) : arrondir un `FLOAT` avec `ROUND` au moment de la requête ne rattrape pas une imprecision déjà introduite au stockage.

7. Sous-requetes

Une sous-requete est une requête imbriquée dans une autre. Elle peut apparaître dans le `WHERE` (filtrer), dans le `FROM` (comme une table temporaire), ou dans le `SELECT` (une valeur calculée par ligne).

```
-- Sous-requete scalaire : clients dont l'age depasse la moyenne
SELECT nom, age
FROM clients
WHERE age > (SELECT AVG(age) FROM clients);

-- Sous-requete avec IN : clients ayant au moins une commande payee
SELECT nom FROM clients
WHERE id IN (SELECT client_id FROM commandes WHERE statut = 'payee');

-- EXISTS : souvent plus rapide que IN sur de gros volumes,
-- s'arrete des la premiere ligne trouvee
SELECT nom FROM clients c
WHERE EXISTS (
    SELECT 1 FROM commandes o
    WHERE o.client_id = c.id AND o.statut = 'payee'
);

-- Sous-requete correlee dans le SELECT : montant total par client, en ligne
SELECT nom,
    (SELECT SUM(montant) FROM commandes o WHERE o.client_id = c.id) AS total
FROM clients c;
```

8. CTE – Common Table Expressions

Une CTE (clause `WITH`) nomme une sous-requete pour la reutiliser et rendre une requete complexe lisible, comme on decoupe un long calcul en variables intermediaires. Une CTE *recursive* permet de parcourir des structures hierarchiques (arbre, graphe) que SQL ne sait pas traverser nativement autrement.

```
-- CTE simple : lisibilite d'une requete en plusieurs etapes
WITH ventes_par_client AS (
    SELECT client_id, SUM(montant) AS total
    FROM commandes
    GROUP BY client_id
)
SELECT c.nom, v.total
FROM clients c
JOIN ventes_par_client v ON v.client_id = c.id
WHERE v.total > 500;

-- CTE recursive : remonter toute la hierarchie d'un employe (ex. organigramme)
WITH RECURSIVE hierarchie AS (
    -- cas de base : l'employe de depart
    SELECT id, nom, manager_id, 1 AS niveau
    FROM employes
    WHERE id = 42

    UNION ALL

    -- cas recursif : le manager du precedent, niveau par niveau
    SELECT e.id, e.nom, e.manager_id, h.niveau + 1
    FROM employes e
    JOIN hierarchie h ON e.id = h.manager_id
)
SELECT * FROM hierarchie;
```

Quand utiliser une CTE

Des qu'une sous-requete est reutilisee plusieurs fois, ou que la requete devient difficile a lire d'un seul bloc. La CTE recursive est le seul moyen standard de parcourir un nombre variable de niveaux (organigramme, categories imbriquees, graphe de dependances).

9. Fonctions fenetrees (window functions)

Une fonction fenetree calcule une valeur *a travers un ensemble de lignes liees a la ligne courante* (la *fenetre*), **sans** regrouper les lignes comme le fait `GROUP BY` : chaque ligne du resultat reste individuelle. C'est l'outil pour repondre a des questions comme « le rang de chaque commande », « le cumul depuis le debut », ou « comparer une ligne a la precedente ».

```
-- Classement des commandes par montant, au sein de chaque client
SELECT client_id, montant,
       RANK() OVER (PARTITION BY client_id ORDER BY montant DESC) AS rang
FROM commandes;

-- Cumul progressif (running total) par client, trie par date
SELECT client_id, montant,
       SUM(montant) OVER (PARTITION BY client_id ORDER BY id) AS cumul
FROM commandes;
```

```
-- Comparer chaque ligne a la precedente / suivante
SELECT client_id, montant,
       LAG(montant) OVER (PARTITION BY client_id ORDER BY id) AS montant_precedent,
       LEAD(montant) OVER (PARTITION BY client_id ORDER BY id) AS montant_suivant
FROM commandes;

-- ROW_NUMBER : numeroter les lignes -- utile pour dedupliquer
SELECT *
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
  FROM clients
) t
WHERE rn = 1;    -- garde seulement la premiere ligne de chaque email en double
```

RANK vs DENSE_RANK vs ROW_NUMBER

Pour des valeurs ex aequo : ROW_NUMBER attribue toujours un numero unique (1, 2, 3...) sans egard aux egalites. RANK laisse un « trou » apres une egalite (1, 2, 2, 4...). DENSE_RANK n'en laisse pas (1, 2, 2, 3...).

10. Types de jointure des ensembles

UNION, INTERSECT et EXCEPT combinent les *resultats* de deux requetes ayant le meme nombre de colonnes et des types compatibles – a ne pas confondre avec les jointures, qui combinent des *colonnes*.

```
-- UNION : combine et supprime les doublons ; UNION ALL les garde (plus rapide)
SELECT nom FROM clients_actifs
UNION
SELECT nom FROM clients_archives;

-- INTERSECT : lignes presentes dans les deux requetes
SELECT email FROM clients
INTERSECT
SELECT email FROM newsletter_abonnes;

-- EXCEPT : lignes de la premiere requete absentes de la seconde
SELECT email FROM clients
EXCEPT
SELECT email FROM newsletter_abonnes;
```

11. Index et performance

Un index est une structure auxiliaire (le plus souvent un arbre B) qui accelere la recherche sur une colonne, au prix d'un espace disque supplementaire et d'un cout a chaque ecriture (l'index doit etre mis a jour). Sans index, chercher une valeur oblige a parcourir la table entiere (*sequential scan*).

```
-- Index simple sur une colonne souvent filtree
CREATE INDEX idx_commandes_client_id ON commandes(client_id);

-- Index composite : utile si les requetes filtrent sur les deux colonnes
-- ensemble (l'ordre des colonnes compte : la premiere doit etre la plus
-- selective ou la plus souvent utilisee seule)
CREATE INDEX idx_commandes_client_statut ON commandes(client_id, statut);
```

```
-- Verifier ce que le moteur fait reellement : lit-il l'index ou la table entiere ?
EXPLAIN ANALYZE
SELECT * FROM commandes WHERE client_id = 42;
```

Quand indexer

Indexer les colonnes utilisees dans **WHERE**, **JOIN ... ON** et **ORDER BY** sur de grandes tables. Les cles primaires et (souvent) les cles etrangeres sont deja indexees automatiquement par le SGBD (a verifier : sur PostgreSQL, une cle etrangere n'est **pas** indexee automatiquement, contrairement a la cle primaire).

Piege courant

Un index existe mais n'est pas utilise si la requete applique une fonction sur la colonne indexee (**WHERE LOWER(email) = ...** n'utilise pas un index simple sur **email**) ou si le filtre porte sur trop peu de lignes distinctes (un index sur une colonne booleenne est rarement utile). Trop d'index ralentissent aussi les ecritures (**INSERT/UPDATE**) : ce n'est pas gratuit, il faut indexer avec intention.

12. Transactions

Une transaction regroupe plusieurs operations en une seule unite *atomique* : soit toutes reussissent (**COMMIT**), soit aucune n'est appliquee (**ROLLBACK**). Indispensable des qu'une operation logique touche plusieurs tables – par exemple, debiter un compte et en crediter un autre ne doit jamais s'arreter a mi-chemin.

```
BEGIN;

UPDATE comptes SET solde = solde - 100 WHERE id = 1;
UPDATE comptes SET solde = solde + 100 WHERE id = 2;

-- Si tout s'est bien passe :
COMMIT;

-- Si une verification echoue (ex. solde negatif) :
-- ROLLBACK;
```

Proprietes ACID

Atomice (tout ou rien) – **C**oherence (les contraintes restent respectees) – **I**solation (les transactions concurrentes ne se perturbent pas) – **D**urabilite (une fois validee, une transaction survit meme a une panne).

13. Erreurs et pieges frequents

Piege	Explication
= NULL au lieu de IS NULL	Ne renvoie jamais TRUE ; utiliser IS NULL / IS NOT NULL.
WHERE sur une table jointe en LEFT JOIN	Transforme silencieusement le LEFT JOIN en INNER JOIN ; filtrer dans le ON.

Colonne non agrégée absente du GROUP BY	Erreur (Postgres) ou valeur arbitraire (MySQL en mode permissif).
HAVING vs WHERE	WHERE filtre avant regroupement, HAVING après.
DELETE/UPDATE sans WHERE	Affecte toutes les lignes de la table.
SELECT * en production	Casse si les colonnes changent ; ramène des données inutiles ; nommer les colonnes explicitement.
Comparer des types incompatibles	'10' = 10 peut fonctionner par conversion implicite mais dégrade les performances (l'index n'est plus utilisable) et masque des bugs.
Oublier les fuseaux horaires	Préférer TIMESTAMPZ à TIMESTAMP dès que l'application a des utilisateurs dans plusieurs fuseaux.

14. Fiche recapitulative

Concept	A retenir
Ordre logique	FROM > JOIN > WHERE > GROUP BY > HAVING > SELECT > ORDER BY > LIMIT .
INNER vs LEFT JOIN	INNER = correspondances uniquement ; LEFT = tout de la table de gauche + NULL .
WHERE vs HAVING	Avant regroupement vs après regroupement.
IN vs EXISTS	EXISTS s'arrête dès la première correspondance, souvent plus rapide sur de gros volumes.
CTE (WITH)	Nomme une sous-requête ; WITH RECURSIVE pour les hiérarchies.
Fonctions fenêtrées	OVER (PARTITION BY ... ORDER BY ...) : calcule sans regrouper les lignes.
Index	Accélère la lecture, ralentit l'écriture ; vérifier avec EXPLAIN ANALYZE .
Transaction	BEGIN / COMMIT / ROLLBACK ; propriétés ACID .
NULL	Valeur inconnue ; toujours IS NULL , jamais = NULL .

Méthode pour écrire une requête complexe

1. Identifier les tables nécessaires et comment elles se relient (clés).
2. Écrire d'abord le **FROM/JOIN**, vérifier avec **SELECT *** que les lignes obtenues sont les bonnes.
3. Ajouter les filtres (**WHERE**), puis le regroupement (**GROUP BY/HAVING**) si nécessaire.
4. Ne sélectionner les colonnes finales et trier (**ORDER BY**) qu'en dernier.
5. Sur une grosse table, vérifier le plan d'exécution (**EXPLAIN ANALYZE**) avant de considérer la requête terminée.